

Exception-Handling

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Was kann schiefgehen?

```
public class ReciprocalCalculator {  
    public static double reciprocalFromFile(String fileName) throws IOException {  
        String content = Files.readString(Path.of(fileName));  
        int value = Integer.parseInt(content);  
        return 1.0 / value;  
    }  
    public static void main(String[] args) throws IOException {  
        double result = reciprocalFromFile("zahl.txt");  
        IO.println("Kehrwert: " + result);  
    }  
}
```

▪ Checked Exceptions

- Müssen deklariert (`throws`) oder gefangen (`try/catch`) werden
- Vom Compiler überprüft
- Beispiele: `IOException`, `FileNotFoundException`

▪ Unchecked Exceptions

- Unterklassen von `RuntimeException`
- Keine Pflicht zur Deklaration oder zum Fangen
- Beispiele: `NullPointerException`, `IllegalArgumentException`, `ArithmeticException`

Important

Merksatz:

- Checked = *erwartbare* Fehlerquellen (z.B. I/O)
- Unchecked = meist *Programmierfehler* / Logikfehler

Fangen und Behandeln von Exceptions mit *Try-Catch* oder *Throws*

Variante A: Exception behandeln

```
public static void main(String[] args) {  
    try {  
        double result = reciprocalFromFile("zahl.txt");  
        IO.println("Kehrwert: " + result);  
    } catch (IOException e) {  
        System.err.println("Fehler beim Lesen der Datei: " + e.getMessage());  
    }  
}
```

Variante B: Exception weiterreichen

```
public static void main(String[] args) throws IOException {  
    double result = reciprocalFromFile("zahl.txt");  
    IO.println("Kehrwert: " + result);  
}
```

Try und mehrstufiges *Catch*

```
public static void main(String[] args) {  
    try {  
        double result = reciprocalFromFile("zahl.txt");  
        IO.println("Kehrwert: " + result);  
    } catch (FileNotFoundException e) {  
        System.err.println("Dateifehler: " + e.getMessage());  
    } catch (NumberFormatException e) {  
        System.err.println("Die Datei enthält keine gültige ganze Zahl.");  
    } catch (ArithmeticException e) {  
        System.err.println("Division durch 0 ist nicht erlaubt.");  
    }  
}
```

Finally und Aufräumen

```
BufferedReader reader = null;  
try {  
    reader = new BufferedReader(new FileReader(fileName));  
    return reader.readLine();  
} finally {  
    if (reader != null) reader.close();  
}
```

Freigeben von Ressourcen: *Try-with-Resources*

Problem bisher

```
BufferedReader reader = null;  
try {  
    reader = new BufferedReader(new FileReader(fileName));  
    return reader.readLine();  
} finally {  
    if (reader != null) reader.close();  
}
```

Freigeben von Ressourcen: *Try-with-Resources*

Problem bisher

```
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader(fileName));
    return reader.readLine();
} finally {
    if (reader != null) reader.close();
}
```

Lösung: try-with-resources

```
try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
    return reader.readLine();
} // reader wird hier automatisch geschlossen
```


Werfen von Exceptions mit *Throws*

```
int div(int a, int b) throws IllegalArgumentException {  
    if (b == 0) throw new IllegalArgumentException("Can't divide by zero");  
    return a / b;  
}
```

Hinweis: throws und checked vs. unchecked

Anti-Beispiele und Best Practices

Anti-Beispiel 1: Leerer catch-Block

```
try {  
    double result = reciprocalFromFile("zahl.txt");  
} catch (IOException e) {  
    // nichts... oder e.printStackTrace()  
}
```

Anti-Beispiele und Best Practices

Anti-Beispiel 1: Leerer catch-Block

```
try {  
    double result = reciprocalFromFile("zahl.txt");  
} catch (IOException e) {  
    // nichts... oder e.printStackTrace()  
}
```

Anti-Beispiel 2: Zu allgemein fangen

```
try {  
    double result = reciprocalFromFile("zahl.txt");  
} catch (Exception e) {  
    System.err.println("Irgendetwas ist schiefgegangen.");  
}
```

- Exceptions trennen **Normalfall** und **Fehlerfall** im *Kontrollfluss*
- Checked vs. Unchecked:
 - Checked = behandeln oder deklarieren
 - Unchecked = keine Pflicht, aber bewusst einsetzen
- `try-catch-finally`:
 - `try`: "gefährlicher" Code
 - `catch`: definierte Reaktion
 - `finally`: Aufräumarbeiten



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

